确定性限价订单簿与撮合系统

复习手册：做了什么、为什么这样设计、怎么证明它是对的

范围：L1 核心撮合 · L2 性能工程 · L3′ 行情与流水线 · L4 下单链路（L5、L6 为未来计划）
仓库状态：main ee03bd5 (2026-11-05) · 生成日期：2026-11-05
所有数字来自 docs/handbook/numbers.yaml（每项注明出处）；带 † 的是墙钟数字。
配套英文版 handbook-en.pdf 使用同一大纲与章节编号。

目录

0 如何使用本手册 3
1 项目一页纸 3
2 L1 核心撮合引擎 4
3 L2 性能工程 6
4 L3′ 行情与流水线 7
5 关键故事：停牌恢复宽限（D-054 / D-055） 8
6 全市场与真实数据 10
7 L4 下单链路与 tick-to-trade 10
8 工程方法 12
9 局限与未验证项（附回答方式） 12
10 精选面试问答 13
11 未来计划：L5、L6（未实施） 14
A 墙钟数字一览 14
B 本机复测与重新生成 16

读法：先读第 0、1 章（两分钟讲完整个项目），再按面试岗位挑章节；第 9、10 章是答问材料。† 的含义见每页页脚。

† 墙钟数字：部分已本机复测，来源见附录 A

2 / 16

0 如何使用本手册

本手册覆盖仓库 ROADMAP 中已完成的全部层级：**L1 核心撮合**、**L2 性能工程**、**L3′ 行情与流水线**、**L4 下单链路**；L5、L6 是未来计划（第 11 章）。 每章按同一个顺序写：**做了什么 → 为什么这样设计 → 怎么证明它是对的**。框起来的段落是可以直接说出口的版本。

30 秒版本

我用 C++20 从零做了一个确定性的限价订单簿和撮合引擎，再在上面接了 Nasdaq ITCH 行情、多线程流水线、策略与事前风控，最后用 OUCH / SoupBinTCP 协议把订单发到一个单线程 epoll 的模拟交易所。 整个项目的原则是"正确性 > 可验证性 > 性能"：每一层都有和朴素参考模型的差分测试，多线程和网络模式都要和单线程、进程内的结果逐位相同，所有性能数字都写明测量环境。

2 分钟版本（按这个顺序讲）

1. **L1**：价格-时间优先的撮合引擎，所有校验在修改之前完成，事件流可以完整重建状态；用参考模型做了 2 000 万 条命令的差分长跑，日志和快照经得起进程被杀。
2. **L2**：先写测量方案再测量；剖析发现会话层每提交一条命令就复制一次全部历史，修成几何扩容后，同一会话交替测量从 4.3–4.6 k 条/s† 到 0.71–1.09 M 条/s†——我会强调这修的是复杂度缺陷，不是"引擎快了几百倍"。
3. **L3′**：ITCH 5.0 解码、按订单重建订单簿、解码线程和订单簿线程之间的无锁 SPSC；多线程结果与单线程逐字节一致；用真实的 Nasdaq 全天数据验证，**真实数据推翻过我的一条规则**（第 5 章）。
4. **L4**：OUCH 4.2 下单、SoupBinTCP 会话、断线重连与防重复报单；网络模式在锁步下与进程内逐位相同；tick-to-trade 的软件路径只有 0.37 µs†；回环多出来的部分取决于机器——云端 VM 上主要是唤醒对端线程，本机复测时这一点没有重现，两种结果我都写了下来。

阅读建议：先读第 0、1 章；面试前一天读第 5、9、10 章。带 † 的数字是墙钟数字（吞吐、延迟、tick-to-trade），含义见页脚；它们都可以用附录 B 的一条命令在本机复测并重新生成本手册。

1 项目一页纸

1.1 做了什么

层级	内容	一句话证据
L1 核心	类型与校验、订单簿、撮合引擎、测试基础设施、回放 / 快照 / 恢复	每个预设 197 个测试；差分长跑 2 000 万 条命令；25 个变异全部被杀死
L2 性能	测量方法、剖析、一个有假设的实验（E1）、基线开关、等价性、内存	同会话交替 A/B；16 个输入两种实现逐字节一致
L3′ 行情	ITCH 5.0 解码、订单簿重建、Feed 状态机、SPSC 流水线、策略风控、全市场	单线程 == 多线程；真实数据 3 个交易日 26 040 只股票·日全部 Live
L4 下单	OUCH 4.2、SoupBinTCP、订单网关、epoll 模拟交易所、tick-to-trade	锁步网络模式与进程内逐位相同；自由运行两端记录可逐位重放

† 墙钟数字：部分已本机复测，来源见附录 A

全仓库：6 个构建预设（GCC / Clang × Debug / Release / ASan+UBSan）每个 501 个测试、零警告，另有两个 TSan 预设各 79 个并发测试；决策记录 D-001 至 D-065。

1.2 整体架构

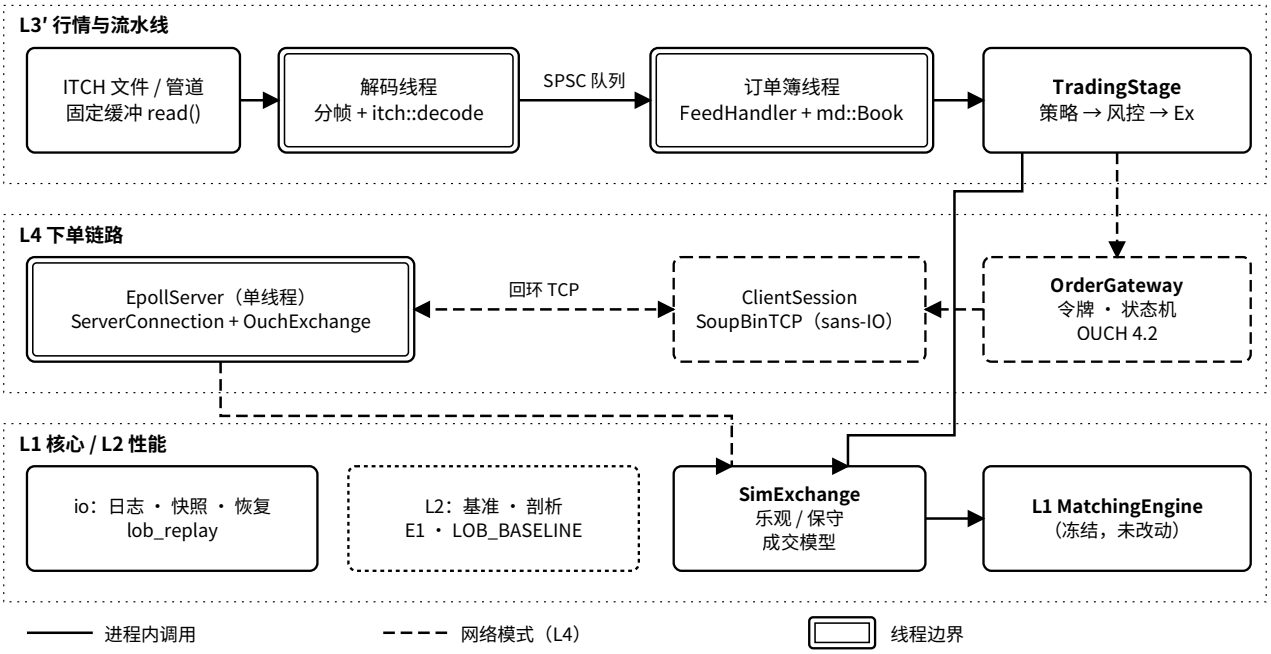


图 1　整体架构。三条带分别是 L3′、L4 与 L1/L2；双线框是独立线程，虚线是 L4 的网络模式。

数据从左上进入：解码线程把文件或管道里的 ITCH 帧解码，经 SPSC 队列交给订单簿线程；订单簿线程重建订单簿、产生行情事件，再交给交易阶段（策略 → 风控 → 交易所）。 交易所有两种：进程内的 `SimExchange`（内部是冻结的 L1 撮合引擎），或经过订单网关、OUCH / SoupBinTCP、回环 TCP 的单线程 epoll 模拟交易所——后者里面仍是同一个 `SimExchange`。

1.3 贯穿全项目的三条原则

- **正确性 > 可验证性 > 性能。** 任何优化都不能以"无法证明结果没变"为代价；性能数字只在说明了环境和方法时才算数。
- **确定性。** 价格用整数 tick；风控的令牌桶用行情时间而不是系统时间；多线程、网络模式都要求与单线程、进程内的输出逐字节相同。确定性让"对拍"成为可能，对拍让正确性可以被证明。
- **先测量后结论，并写下没验证的东西。** 例如：每个墙钟数字都注明测于哪台机器；云端得出的"回环主要是唤醒代价"在本机复测时没有重现，两个结果都保留；掉电持久性"未验证"；L1 的撮合引擎每次挂单仍会分配内存，所以"零分配"只针对行情热路径。

2 L1 核心撮合引擎

2.1 类型与错误处理

- **价格是 `int64` 整数 tick**（D-001）：浮点无法精确相等，累加不确定。**数量是有符号 `int64`**（D-003）：无符号下 `remaining − fill` 的错误会静默回绕成巨大正数，有符号则能被校验和 UBSan 发现。

† 墙钟数字：部分已本机复测，来源见附录 A

- **运行期输入错误返回拒单码，配置错误抛异常**（D-004）：非法订单是常态不是异常，热路径不抛不分配；配置非法是部署错误，构造时立即失败。
- **配置约束保证不溢出**（D-006）：`max_live_orders × max_order_quantity ≤ INT64_MAX`，所以任何累加都不需要逐次检查——溢出在配置时就被排除了。

2.2 订单簿（M2）

基线结构是标准容器：两侧各一个 `std::map` 价格层，层内 `std::list` 保持 FIFO，`unordered_map` 从订单号到位置（D-007）。先建立可证明正确的基线，侵入式链表或对象池作为 L2 的实验候选，而不是预先引入。 订单簿**不含撮合逻辑**，只负责按价格-时间优先存放挂单；`find_invariant_violation()` 在测试中每一步都重新推导全部结构事实（层总量、FIFO、索引一致、永不交叉）。

2.3 撮合引擎（M3）

- **价格-时间优先**，成交价是 maker 的价格；一次可以扫过多个价位。
- **单一 seq 时钟**（D-020）：每个事件（含拒单）都取一个递增的 seq，挂单的时间优先级就是它被受理时的 seq。一条时钟同时给出事件全序和队列顺序，审计时能从事件流直接推出队列。
- **两阶段处理**（D-019）：准备阶段只读——全部校验、seq 溢出检查、事件容量预留；提交阶段不分配、不抛出，唯一可能失败的是挂剩余量时的内存分配，此时发出终结事件后重抛（D-024），事件流始终完整。
- **`BOOK_FULL` 的判定**（D-021）：先预演，只有"有剩余且簿满"时才拒绝。可以证明：被拒的订单一定完全不会成交，所以不存在"成交了一半再被拒"的副作用。

2.4 测试基础设施（M4）

正确性的主证据不是手写用例，而是**差分测试**：一个刻意写得朴素、显然正确的参考引擎，与真实引擎跑同一条随机命令流，逐事件、逐快照比较。

- 长跑矩阵：10 个工作负载画像的长跑共 2 000 万 条命令，每 1 000 条全量比较；L1 门槛是 ⩾ 20 个种子 × ⩾ 10^4 条命令。
- 事件审计器：从事件流独立验证数量守恒（受理 = 2 × 成交 + 剩余 + 撤单），用 128 位整数累加避免审计器自己溢出。
- 故障注入：替换全局 `operator new`，在第 k 次分配时失败，覆盖每一个分配点；长序列中 2% 的命令武装故障后继续运行。
- 变异测试：故意改坏引擎（例如"按 taker 价格成交""BOOK_FULL 不算被腾出的位置"），确认测试会失败——25 个变异全部被杀死。
- 跨编译器：GCC 与 Clang 的覆盖、状态、故障行逐字比较完全相同。

2.5 回放、快照与恢复（M5）

- **文本格式 v1**（D-034）：严格、规范、带版本号，拒绝多余空格和前导零，所以"逐字节相同"等价于"语义相同"，可以写 Golden 文件（5 个手写场景 + 1 个 2 000 条命令的固定种子场景）。
- **日志 = 每条命令一个带校验和的批次，事件为真源**（D-035）：恢复时按事件折叠，不重新执行命令——即使历史中有内存分配失败，也能精确恢复。
- **尾部残缺与损坏的区分**（D-036）：写到一半的尾部丢弃并截断；校验失败但后面还有数据则判损坏、拒绝自动恢复。对最后一批次每个字节做 5 种修改：417 次报告损坏、4 次整批丢弃，**0 次**接受被修改的数据。

† 墙钟数字：部分已本机复测，来源见附录 A

- **提交协议**（D-038）：先持久化后确认；持久化失败即"中毒"，只能从日志恢复。真实进程被 `SIGKILL` 后重启续跑，结果与不中断一致。

2.6 怎么证明 L1 是对的

性质	证据
撮合语义	每条语义（价格优先、同价 FIFO、部分成交、跨层扫单、各种撤单、每种拒单、边界值）至少一个具名测试
与参考模型一致	差分长跑 2 000 万 条命令；CTest 里每次构建另跑 20 个种子 × 5 000 条
确定性	同一输入回放两次、GCC 与 Clang 两种构建，事件流与快照哈希相同；Golden 文件逐字节一致
异常安全	逐分配点故障注入；ASan / UBSan / LSan 下长跑
崩溃一致性	逐字节截断、写失败注入、真实进程 `kill -9` 后重启

3 L2 性能工程

3.1 测量方法（先写方案，再测量）

- 方案在任何测量之前写定（BENCHMARK §1–§6）：数据生成和校验在计时区外；吞吐和逐操作延迟分开测；每次重复从同一状态开始并校验状态哈希。
- **只在同一会话交替运行的 A/B 下下结论**（D-043）；进程间范围重叠即"噪声内"，本环境 ±10% 以内的差异不可分辨。跨会话的数字曾在代码完全没变的情况下相差 2–23%——这正是这条规则的来由。
- 环境：作者的 WSL2 虚拟机没有硬件性能计数器，缓存与分支只能用 cachegrind 模拟，并始终标注为模拟值。

引擎在 7 个画像上每秒数百万到两千万条命令，p99 约 0.2–1.2 µs†；常规混合负载 11.6 M 条/s†，p50 / p99 为 91 ns† / 410 ns†。

3.2 剖析结论

分配器占引擎指令的 25–35%，是最大的单项；但池化要改 M2 的容器和移动语义，风险与收益在 L2 内不成比例，留作后续任务（D-044）。用不改代码的 glibc 调参探针（P1）测了分配器慢路径的敏感度：指令数 −8.9%，墙钟在噪声内——**一个如实报告的空结果**。

3.3 E1：一个复杂度缺陷的故事

- **现象**：会话（Session）吞吐随流长度约 1/n 下降；`perf` 95% 的样本在 `submit` 的复制循环里。
- **假设**：提交记录用 `reserve(size + 1)` 精确扩容，每提交一条就把全部历史复制一次——整个会话 $O(n^2)$。L1 的事件缓冲区早就因为同样的问题改成了几何扩容（D-022），M5 没有沿用。
- **改动**：容量不足时 `reserve(max(size + 1, 2 × capacity))`，改动 +7 / −2 行；`reserve` 仍在任何效果之前，提交点之后仍不分配。
- **结果**（同会话交替 5 对）：非持久会话 4.3–4.6 k 条/s† → 0.71–1.09 M 条/s†；回放延迟 p50 152 µs† → 0.78 µs†；持久模式与引擎对照在噪声内。

† 墙钟数字：部分已本机复测，来源见附录 A

6 / 16

> **面试时这样说**
> 倍数取决于流长度——缺陷成本随长度平方增长——所以我不说"快了 约 150–250 倍†"。正确的说法是：每条命令的会话成本从"随已提交命令数线性增长"变成了常数；撮合引擎和订单簿一行没改。

3.4 基线开关、等价性与内存

- **基线实现保留为编译期选项**（D-045）：`LOB_BASELINE=ON` 编译冻结的基线源文件，`OFF` 编译优化版；优化版的源码和生成代码都不受影响。
- **等价性**：16 个输入，两种实现的事件流、日志、快照逐字节相同。
- **内存与预期相反**：按容量估算，优化版应该占用更多；实测峰值 RSS 反而更低（mixed −1 880 KiB（−5.9%），deep_book −5 464 KiB（−15.2%））。已验证两条机制（基线扩容时新旧块并存；预留但未写入的页不计入 RSS），但它们解释不了全部差值——**写明"仍未解释"**。
- 一条命令复现全部对比：`scripts/l2_reproduce.sh`（73 分钟）。

4 L3′ 行情与流水线

4.1 ITCH 5.0 解码与订单簿重建

- 解码 11 种消息（系统事件、目录、新单、成交、撤单、删单、改单、交易状态、集合竞价成交等），其余规范类型按精确长度跳过；零拷贝，非法输入返回错误码而不是崩溃。
- **为什么不复用 L1 的订单簿**（D-046）：L1 只能成交队首、没有部分撤单、要求 seq 递增；而 ITCH 是"交易所告诉你成交了哪一单"，可以指向任意订单。重建器只照做交易所说的事，不自己撮合。
- `md::Book` **先校验后修改**：一条消息被拒时订单簿保持原样，这样才能精确断言"拒绝没有副作用"。
- 读取用固定缓冲区而不是 `mmap`：峰值内存只由配置决定，回放 33 MB 到 984 MB 的输入都是 ≈ 47 MB，还能从管道读（边下载边解压边回放）。

4.2 FeedHandler 状态机

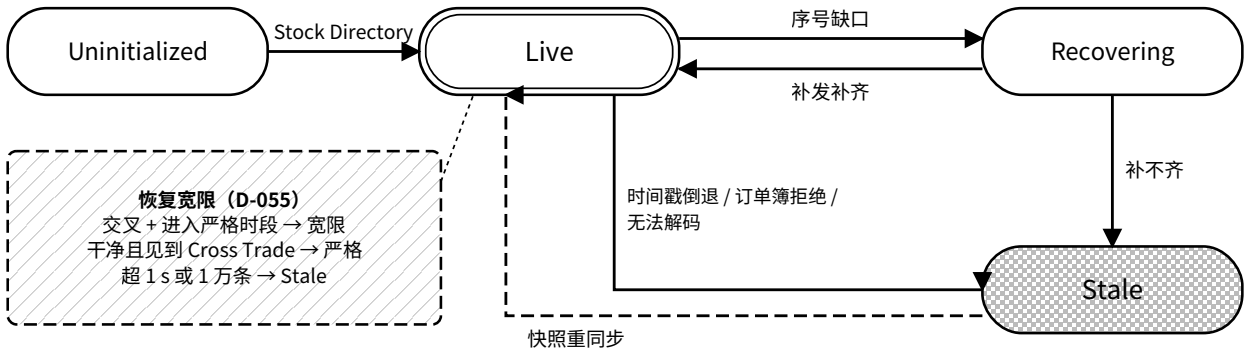


图 2　每只股票一个 FeedHandler。非 Live 时不发布订单簿更新；恢复宽限（虚线框）是 Live 内部的子状态，见第 5 章。

缺口检测靠**序号**而不是时间戳：安静的时段是正常的，时间戳只要求不倒退。任何无法解释的输入都让该股票进入 Stale，Stale 期间下游只看到状态事件，绝不使用一本可能错误的订单簿。

† 墙钟数字：部分已本机复测，来源见附录 A

4.3 SPSC 与流水线：多线程仍然确定

- 单生产者单消费者环形队列：索引是不回绕的 64 位计数器，满 / 空判断不浪费槽位；生产者和消费者的索引各占一个缓存行；`tail` 发布用 release、读取用 acquire——看到新 tail 就一定看到完整的元素。
- **无损背压**（D-048）：队列满时生产者自旋，绝不丢弃。
- **确定性**：解码函数和订单簿线程对象在单线程模式和流水线模式中是同一份代码；两种模式的事件流、订单簿、交易摘要逐字节相同（GCC、Clang、TSan 下都测）。

4.4 延迟瀑布与零分配

- 每跳用 TSC 打时间戳，启动时校准并测出"读两次时钟"的开销，从每个样本中扣除（D-049）。解码一跳 p50 / p99 18 ns† / 67 ns†，订单簿一跳 12 ns† / 237 ns†，单线程回放 9.7 M 条/s†。
- **饱和模式下的"队列一跳"是排队时间而不是交接时间**（约 1.4 ms† = 队列容量 × 每条处理时间）；要看跨核交接必须看限速模式：p50 104 ns†，p99 在重复之间相差 2.1–40 倍†——虚拟化噪声，只能当上界。
- 热路径零分配：专用测试替换全局 `operator new`，每个线程在热循环开始时"布防"，期间任何分配即失败；另有故意分配的负对照证明它能抓到。

4.5 策略、风控与模拟成交

- 策略接口用 CRTP 静态分发；与虚函数版本实测 10.8† vs 11.2† ns，**分辨不出差别**——不声称没测出来的收益。
- 事前风控按固定顺序检查：熔断 → 交易阶段 → 参考价 → 乌龙指数量 → 乌龙指金额 → 价格带 → 最大持仓 → 令牌桶；每次检查 ≈ 6 ns†。撤单永不拦截；熔断是粘滞的。令牌桶用**行情时间**补充，风控因此是输入的纯函数。
- 模拟交易所把通过风控的订单送进**未改动的 L1 撮合引擎**；ITCH 报告的市场成交被转换成"合成主动单"去撮合我们的挂单。两个成交模型同时运行：乐观（假设我们在队首）与保守（排在当时可见队列的末尾），对同一订单流构成上下界（D-053）。

4.6 CI 的性能门槛：指令数而不是时间

共享的 CI 机器上墙钟时间每次相差几十个百分点；cachegrind 的指令数对固定输入是完全可复现的。门槛：decode / book / trading 三个负载 33.5 M / 121.3 M / 266.3 M 条指令，超过 +2% 即失败；编译器或 valgrind 版本不同时拒绝比较（D-051）。它抓的是"多做了工作"，抓不到缓存未命中——墙钟基准只在本地跑。

5 关键故事：停牌恢复宽限（D-054 / D-055）

5.1 交易阶段感知（D-054）

真实交易所的订单簿在集合竞价、停牌、LULD 暂停、盘前盘后都**可以合法地锁定或交叉**。规则：只在常规时段（System Event 'Q' 到 'M'）且个股处于交易状态 'T' 时强制"永不交叉"；其他时段允许、计数并标记，策略不报价，风控拒新单但不熔断。

† 墙钟数字：部分已本机复测，来源见附录 A

8 / 16

5.2 真实数据推翻了规则

用 Nasdaq 2019-01-30 全天数据回放 SXTC（当天两次 LULD 暂停）。D-054 的规则在恢复交易的那一刻把它判成了 Stale——**一个正确的订单簿被判为错误**。

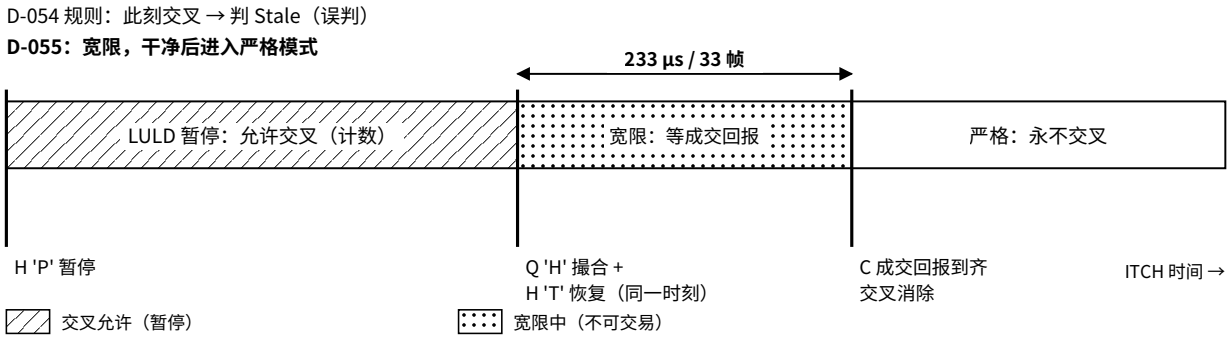


图 3　SXTC 第一次恢复（示意）。撮合（Q 'H'）和恢复（H 'T'）在同一时刻到达，参与撮合的各订单的成交回报随后才到。

原因是到达顺序：暂停期间 10:27:49 → 10:32:49 订单簿交叉；恢复时交易所先撮合并宣布恢复，但参与撮合的每一笔订单的成交回报要在之后 33 帧 / 233 µs（第二次恢复 37 帧 / 264 µs）才陆续到达。在这段时间里订单簿"看起来"交叉。

5.3 宽限规则与它的边界

进入严格时段时如果订单簿仍交叉，不立即判 Stale，而是进入**恢复宽限**：

- 宽限中订单簿保持非严格、事件带宽限标记、**不可交易**；
- 订单簿干净 **且** 见到了该股票的 Cross Trade（撮合证据）→ 进入严格模式；
- 超过 1 秒（ITCH 时间） 或 10 000 条 该股票的消息仍未解除 → Stale；
- 宽限中再次暂停 → 取消宽限，下一次恢复需要新的撮合证据。

> **追问：宽限会不会掩盖真正的错误?**
>
> 三个约束：必须看到 Cross Trade 作为证据（没有撮合的交叉 1 秒后照样判 Stale）；有时间和消息数两个上限；宽限中不可交易。上限按实测 233–264 µs 留了 约 4 000 倍 余量；3 个交易日 28 次宽限全部在上限内解除——但每次的用时还没统计，所以它仍是一个需要更多数据复审的参数。

5.4 证据

- 10 个具名测试，包括按 SXTC 真实顺序手写的完整事件流、"恰在上限不超时、超 1 ns 超时"、宽限中再次暂停、其他股票的 Cross Trade 不算。
- 第二次回放：SXTC 全天 Live，两次宽限均由成交回报解除；单线程与流水线两种模式哈希一致。
- 指令数门槛拦下过第一版实现：超时检查内联在热路径中使 book 负载 +2.94%；移到不内联的冷路径函数后 +1.10%。

† 墙钟数字：部分已本机复测，来源见附录 A

> **面试时这样讲（约 1 分钟）**
> 我先根据规范写了一条看似合理的规则：常规交易时段订单簿不允许交叉。拿真实数据一跑，一只停牌恢复的股票在恢复的那一刻被判成错误。我没有放宽规则了事，而是去看消息的到达顺序：撮合和恢复同时到，成交回报晚 233–264 µs。于是我设计了一个有界的宽限——要求撮合证据、有时间和消息数上限、宽限中不交易——再用 3 个交易日的全市场数据验证：26 040 只股票·日没有一只误判。这是"先测量、再定规则、并写明依据"的例子。

6 全市场与真实数据

6.1 全市场订单簿（D-056）

- 每只股票一个 FeedHandler，所有股票的订单共享一个订单池（订单号全市场唯一）；按股票编号取模分片到 N 个工作线程。
- **为什么取模而不是按负载动态分配**：取模让"哪只股票在哪个线程"只取决于输入，结果可复现、可对拍；动态迁移会让确定性和测试都更难。
- 汇总哈希是逐股票摘要之和（与遍历顺序无关），所以单线程与 N 分片可以直接比较。

6.2 真实数据结果（Nasdaq 2019-01-30 全天，流式不落盘）

项	结果
股票数 / 最终状态	8 713 只，全部 Live，0 Stale
峰值在途订单	1 742 866
单线程 vs 4 分片	逐股票哈希文件逐字节相同，汇总哈希 `0xf9bcda01d2d8d01b`
吞吐	单线程 1.35 M 帧/s†，4 分片 3.11 M 帧/s†（线程数多于 vCPU，未线性扩展）
多日验证	3 个交易日共 26 040 只股票·日全部 Live；28 次恢复宽限全部解除

单只股票的回放（AAPL）：11.2 GB、3.68 亿 帧，峰值内存 98.7 MiB，0 次 Stale。

6.3 全市场模式零分配（D-058）

全市场模式里股票在目录消息到达时才创建、价位数组按需增长——两处都会分配。做法：先跑一遍写出每只股票的价位峰值画像（最大 4 227），回放前按"峰值 × 2 倍"预创建并**封顶**；超出的股票判 Stale，绝不分配。内存 45–51 MB，而统一按最大值预留需要 约 1.8 GB。代价写明：画像来自历史日期，某只股票当天可能超出，结果是可见的 Stale 而不是隐蔽的分配。

7 L4 下单链路与 tick-to-trade

7.1 分层：OUCH 与 SoupBinTCP

- **SoupBinTCP** 只负责把交易所发出的每条消息不丢、不乱地送到客户端：登录、隐式序号、断线后按"下一期望序号"重放、1 秒心跳、15 秒超时、会话结束。它**不保证**客户端发出的消息。
- **OUCH 4.2** 负责订单语义，而它的每条入站消息都可以**安全重发**：重复令牌被交易所忽略，撤单写的是"撤后剩余量"而不是减量。入站幂等，所以会话层只需保证出站——这是整个恢复设计的支点。

† 墙钟数字：部分已本机复测，来源见附录 A

10 / 16

· 两层都写成 **sans-IO** 状态机：输入是"字节 + 当前时间"，输出是"要发的字节 + 事件 + 下一个超时点"，每个状态转换都能用手写字节和假时钟测试，不需要网络、不需要 sleep。

7.2 订单网关

· 三套编号：策略的 `client_id`；线上唯一的 OUCH 令牌（改单换新令牌）；交易所的订单参考号。令牌 = 日期 + 进程启动序号 + 计数，启动序号持久化，进程重启后也不会复用当天的令牌。

· 每单状态机：PendingNew → Live → Filled / Canceled / Rejected；"已请求撤单"时仍可能收到成交（撤单与成交在线上赛跑）。任何解释不了的回报（未知令牌、超量成交、终态后的消息）→ 网关"不一致" → 熔断并撤全部，**宁可停止交易也不猜持仓**。

· 断线重连：撤单 / 改单原样重发（幂等）；新单默认不重发（它基于断线前的行情，已经过时）。"重放何时结束"规范没有给高水位，用规范自己的信号：服务端只在 1 秒没发任何东西时才发心跳，登录后第一个心跳就说明重放发完了。

7.3 模拟交易所

单线程　epoll；交易所**自己跑一份行情订单簿**（保守成交模型要读交易所看到的排队量）；撮合仍是未改动的 L1 引擎。实现了规范里的改单四种结局、部分撤单、交易所侧限流（超速的新单 / 改单回规范中的拒单原因 'x'，**撤单不限流**）和断线撤单（编入序号，重连后在重放里能看到）。

7.4 网络模式怎么证明是对的

回报什么时候到达取决于线程调度，同一份输入可能有不同的交错。分两件事证明：

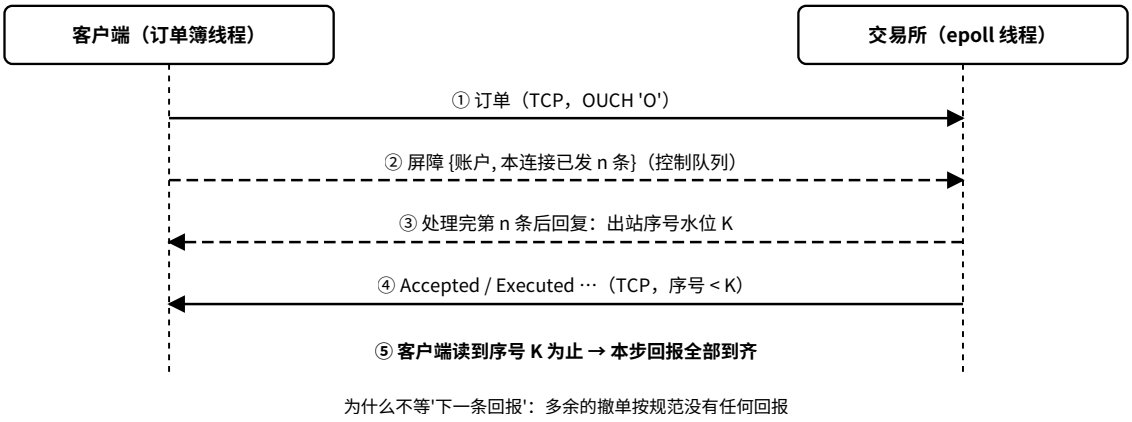


图 4　锁步模式。屏障按"本连接已发的入站消息数"计数，回复携带出站序号水位。

· **锁步**证明"网络栈是透明的"：每一步之后等到静默，回报的交付顺序与进程内完全一样。门槛：订单 / 成交 / 持仓哈希与进程内**逐位相同**——两种成交模型、单线程与多线程流水线都测了。

· **自由运行**不比哈希，查不变量：两侧敞口不超过持仓上限、没有重复令牌、状态机合法、排空后持仓 / 现金 / 在途 / 序号逐项对账；并把两端实际看到的交错**记录下来，离线重放必须逐位复现**。注入三次断线和交易所限流时全部成立。

· 测试本身也被测试：故意去掉锁步中的等待，两个锁步测试失败；故意少记一次投递，重放测试失败。

7.5 tick-to-trade

定义：从 ITCH 帧开始解码到 OUCH 字节交给内核（`send()` 返回），只统计产生了订单的帧，扣除读时钟开销。**先发后做**：本帧订单发出之后才转发行情、才读回报，窗口里没有别的系统调用。

† 墙钟数字：部分已本机复测，来源见附录 A

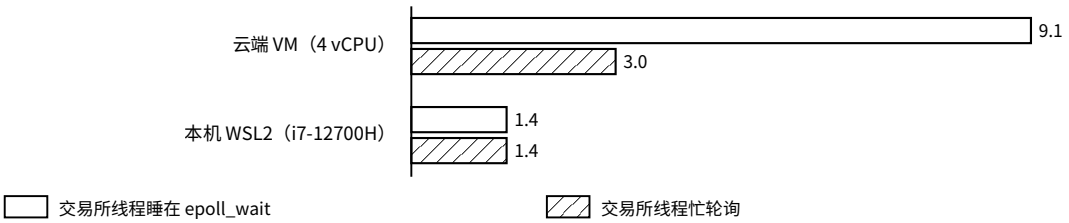


图 5　同一个回环测量在两台机器上的 p50。白色：交易所线程睡在 epoll_wait；斜线：交易所线程忙轮询。云端忙轮询明显降低它，本机没有差别。

- 进程内 tick → 本帧第一条消息编码完成：0.37 µs† ——软件路径本身。
- 本机回环：tick → 第一条消息 `send()` 返回 1.4 µs†，交易所线程忙轮询时 1.4 µs†；一次 `send()` 约 0.88 µs†；客户端 `send()` → 交易所解码 171 µs†；往返 447 µs† / 395 µs†。
- **同一个实验，两台机器，两个结论。** 云端 VM 上忙轮询把第一条消息从 9.1 µs† 降到 3.0 µs†、往返从 28 µs† 降到 18 µs†，所以当时的结论是"回环的主要代价是唤醒对端线程"。本机复测时忙轮询前后没有差别，这个结论因此**只对云端 VM 成立**。得出它的方法仍然有效：控制变量，先把窗口里不属于下单路径的工作（读回报、转发行情的 eventfd 写）逐项移出，再用"新单 vs 本帧第一条消息"的差值估出一次 `send()`，最后用忙轮询实验检验。
- **本机往返慢的原因还没解释。** 已知它不是跨核计时误差（往返在客户端同一个核上计时），不是交易所线程睡眠（忙轮询不改变它），也不在软件路径。下一步是在同一台机器上跑一个最简单的回环 TCP ping-pong，把平台本身的回环延迟和本项目的代码分开。
- 这些不是线到线数字：没有网卡、交换机、真实网关，也没有内核旁路；只和自己的上一版比较。

8 工程方法

- **CI**：6 个预设零警告 + 两个 TSan 预设 + cachegrind 指令数门槛；`scripts/check.sh` 是统一入口。
- **负对照**：每个子阶段都故意改坏代码，确认测试失败后恢复。一次负对照报告"未检出"，追查发现变异体因 `-Werror` 根本没编译成功、跑的是旧二进制——之后每次先确认构建成功。
- **决策记录**：65 条决策都写明选项、选择、代价与复审条件；设计偏离（例如同账户第二次登录由"拒绝"改为"接管"）也记录在案。
- **数字纪律**：每个数字写明环境与出处；本手册的数字全部来自 `numbers.yaml`，墙钟数字带 †。
- **AI 协作披露**：AI 参与了设计与实现，分工与作者决策逐阶段记录在 `AI_USAGE.md`。面试中被问到时如实说明，并强调验证由测试和可复现的证据承担。

9 局限与未验证项（附回答方式）

局限 / 未验证	面试时怎么回答
墙钟数字来自两台机器（本机 WSL2 与云端 VM），部分只在云端测过	"每个数字都写了机器和日期，附录 A 有出处，一条命令复测。本机复测推翻过我的一个云端结论，所以我不把一台机器上的结论推广到另一台，跨会话的数字也不拿来做归因。"
本机回环往返慢的原因未解释	"我知道它不是计时误差，也不在我的软件路径；下一步用最简单的 ping-pong 测平台本身的回环延迟。在那之前我不给它找解释。"

† 墙钟数字：部分已本机复测，来源见附录 A

12 / 16

局限 / 未验证	面试时怎么回答
回环不是线到线；没有网卡、交换机、内核旁路	"回环说明的是把用户态程序接上内核 socket 的代价；真实线到线要在交换机镜像口用硬件时间戳、两端 PTP 同步。我只和自己的上一版比。"
掉电与操作系统崩溃下的持久性未验证	"测过进程被杀、逐字节截断和写失败；掉电依赖文件系统正确实现 fsync 与 rename，我没有条件测试，所以没有声称。"
L1 撮合引擎每次挂单仍分配内存	"零分配的保证只覆盖行情热路径；引擎池化需要改冻结的 L1，风险和收益不成比例，记录为后续任务。"
成交模型不模拟我们自己的市场冲击	"乐观和保守两个模型给出同一订单流下的上下界；闭环策略下两者的总量不再有序，这一点在测试里被发现并写进了决策。"
恢复宽限的上限需要更多数据复审	"现在有 3 天 28 次宽限全部解除；每次的用时分布还没统计，所以上限是带余量的经验值，不是定论。"
进程重启后不接管上一个进程的在途订单	"令牌在重启后不重复已经保证；接管需要按令牌前缀识别旧实例，是明确的下一步。"
E1 后的内存差值没有完全解释	"两条机制已经验证，但它们解释不了全部差值；我把它写成'仍未解释'，而不是凑一个说法。"

10 精选面试问答

Q1 为什么要确定性？代价是什么？ 确定性让同一输入的结果可比较：差分测试、单线程对拍多线程、进程内对拍网络都依赖它。代价是不能用系统时间、不能依赖调度顺序（风控用行情时间补令牌；多线程只用 SPSC 传递有序数据）。

Q2 SPSC 的 memory order 为什么这样选？ 生产者写完元素后用 release 发布 tail，消费者用 acquire 读 tail，所以看到新 tail 就看到完整元素；各自读自己的索引用 relaxed。没有第三方观察两个索引的相对顺序，不需要 seq_cst。

Q3 多线程了，结果怎么还能和单线程逐字节一样？ 两种模式用同一个解码函数和同一个订单簿线程对象；队列无损、保序；状态只在一个线程里修改。测试在 GCC、Clang 和 TSan 下比较事件流与快照哈希。

Q4 怎么证明撮合引擎是对的？ 朴素参考模型 + 随机命令流的差分测试（2 000 万 条），事件审计器独立验证守恒，变异测试证明测试能抓到错误，故障注入覆盖每个分配点。

Q5 你做过的最好的一次性能优化是什么？ E1：不是"引擎快了"，而是一个 $O(n^2)$ 的复杂度缺陷——剖析定位、写下假设、同会话交替 A/B 验证，并如实说明倍数随流长度变化。另一个值得讲的是空结果：分配器调参在墙钟上分辨不出收益，我没有采纳。

Q6 CI 为什么用指令数做性能门槛？ 共享机器上墙钟抖动几十个百分点，指令数对固定输入完全可复现；它抓"多做了工作"的回归，抓不到缓存效应，所以墙钟基准只在本地跑。门槛曾拦下宽限超时检查内联在热路径上的那一版。

Q7 讲一个真实数据推翻你假设的例子。 第 5 章：停牌恢复时撮合与恢复同时到、成交回报晚 233–264 µs；规则把正确的订单簿判错，于是设计了有证据、有上限、宽限中不交易的恢复宽限，并在 26 040 只股票·日上验证没有误判。

Q8 全市场为什么按股票编号取模分片？ 分配只取决于输入，可复现、可对拍；动态迁移依赖调度。代价是负载可能不均，热门股票集中时可以离线按历史量生成一张固定映射表，仍然确定。

† 墙钟数字：部分已本机复测，来源见附录 A

Q9 断线重连后怎么知道哪些单子到了交易所？ 用旧会话加下一期望序号重新登录，交易所重放断线期间的全部回报；撤单、改单原样重发（幂等）；新单默认不重发；登录后第一个心跳说明重放结束，此时仍无回音的新单必然没到过交易所。

Q10 加了网络之后时序不确定，怎么证明结果是对的？ 锁步证明网络栈透明（与进程内逐位相同；屏障用出站序号水位，因为多余的撤单没有回报）；自由运行查不变量并记录两端的交错，离线逐位重放——出了问题可以带着记录回来单步调试。

Q11 你的 tick-to-trade 是多少？说明了什么？ 先给定义和口径，再给数字：本机进程内 0.37 µs†，回环 1.4 µs†，往返 447 µs†。最值得讲的是它推翻了我自己的结论：云端忙轮询把回环从 9.1 µs† 降到 3.0 µs†，本机却没有差别，所以"唤醒成本"只是那台机器上的事实。它不是线到线，只适合和自己比。

Q12 如果重来，你会改什么？ 更早把真实数据接入验证（规则应该先被数据检验）；更早建立按"新单 vs 第一条消息"之类的控制变量测量；订单簿节点池化和多实例订单接管列为下一步。

11 未来计划：L5、L6（未实施）

项目范围到 L4 为止（D-065）。以下两项只作为未来方向记录：

- **L5 UDP 组播与 A/B 双路仲裁**：交易所把同一个 MoldUDP64 序号流在两条独立路径上各发一遍；接收端按消息序号去重、先到先用、严格按序交付；两路都缺时先等一个有界窗口，再向重传服务请求最小范围；补不齐时交给现有的恢复状态机（Recovering → Stale），不另做一套恢复。门槛：任何可补齐的故障组合下，交付给流水线的消息序列与直接读文件逐字节相同。已有设计草案（`docs/design/L5_DESIGN.md`，未实现）。
- **L6 微观结构信号研究**：从重建的订单簿导出特征（列式文件、与订单簿哈希绑定），做样本内外检验，报告不宣称交易收益。没有设计文档。

A 墙钟数字一览

下表是本手册中全部带 † 的数字、它们目前的测量来源与出处；其余数字的出处见 `docs/handbook/numbers.yaml`。

测于：author WSL2, 2026-10-22

数字	当前值	出处
撮合引擎吞吐（mixed）	11.6 M 条/s	BENCHMARK.md §10
撮合引擎延迟 p50（mixed）	91 ns	BENCHMARK.md §10
撮合引擎延迟 p99（mixed）	410 ns	BENCHMARK.md §10
撮合引擎 p99（7 个画像）	0.2–1.2 µs	BENCHMARK.md §11

测于：author WSL2, 2026-10-21

数字	当前值	出处
E1 前：非持久会话吞吐	4.3–4.6 k 条/s	DECISIONS.md D-042; BENCHMARK.md §9
E1 后：非持久会话吞吐	0.71–1.09 M 条/s	DECISIONS.md D-042; BENCHMARK.md §9
E1 前：回放延迟 p50	152 µs	BENCHMARK.md §12

† 墙钟数字：部分已本机复测，来源见附录 A

14 / 16

数字	当前值	出处
E1 后：回放延迟 p50	0.78 µs	BENCHMARK.md §12
E1 倍数（随流长度变化）	约 150–250 倍	BENCHMARK.md §9

测于：本机（Intel i7-12700H，20 CPUs，WSL2），2026-11-05

数字	当前值	出处
ITCH 回放吞吐（单线程）	9.7 M 条/s	BENCHMARK.md §13.1
解码一跳 p50	18 ns	BENCHMARK.md §13.1
解码一跳 p99	67 ns	BENCHMARK.md §13.1
订单簿一跳 p50	12 ns	BENCHMARK.md §13.1
订单簿一跳 p99	237 ns	BENCHMARK.md §13.1
跨线程交接 p50（限速）	104 ns	BENCHMARK.md §13.1
风控单次检查	≈ 6 ns	BENCHMARK.md §14.1
CRTP 策略调用（ns）	10.8	BENCHMARK.md §14.1
虚函数策略调用（ns）	11.2	BENCHMARK.md §14.1
进程内 tick → 第一条消息编码	0.37 µs	BENCHMARK.md §15.1
回环 tick → 第一条消息 send()	1.4 µs	BENCHMARK.md §15.1
同上，交易所线程忙轮询	1.4 µs	BENCHMARK.md §15.1
往返 tick → Accepted	447 µs	BENCHMARK.md §15.1
往返，交易所线程忙轮询	395 µs	BENCHMARK.md §15.1
客户端 send() → 交易所解码	171 µs	BENCHMARK.md §15.1
饱和模式的队列一跳	约 1.4 ms	BENCHMARK.md §13.1
交接 p99 的重复间差异	2.1–40 倍	BENCHMARK.md §13.1
一次回环 send()	约 0.88 µs	BENCHMARK.md §15.1

测于：cloud VM 4 vCPU, 2026-11-04

数字	当前值	出处
全市场吞吐（单线程）	1.35 M 帧/s	MODULE_NOTES/L3.md
全市场吞吐（4 分片）	3.11 M 帧/s	MODULE_NOTES/L3.md

测于：cloud VM 4 vCPU, 2026-11-05

数字	当前值	出处
云端：回环 tick → 第一条消息 send()	9.1 µs	BENCHMARK.md §15
云端：同上，交易所线程忙轮询	3.0 µs	BENCHMARK.md §15
云端：往返 tick → Accepted	28 µs	BENCHMARK.md §15

† 墙钟数字：部分已本机复测，来源见附录 A

数字	当前值	出处
云端：往返，交易所线程忙轮询	18 µs	BENCHMARK.md §15

B 本机复测与重新生成

在仓库根目录运行（WSL2 / Linux；首次需要 `sudo apt install python3-venv fonts-noto-cjk fonts-dejavu-core poppler-utils valgrind`）：

```
docs/handbook/remeasure.sh                  # 复测并重新生成两份 PDF
docs/handbook/remeasure.sh --itch FILE      # 另外用真实 ITCH 文件复测全市场吞吐
docs/handbook/remeasure.sh --l2             # 另外重跑 L2 基线对比（约 73 分钟）
docs/handbook/remeasure.sh --from-log LOG   # 不测量：导入手动运行的基准日志
docs/handbook/build.sh                      # 只重新生成 PDF
```

`remeasure.sh` 构建 Release、绑核运行 `lob_bench`、`lob_itch_bench` 与 `lob_t2t`（各重复多次取中位数），把结果、日期和环境写回 `numbers.yaml`，再调用 `build.sh`。页脚的 † 说明由各数字的测量来源自动生成：全部本机复测后变为"本机实测"及日期；附录 A 按机器列出每个数字的来源。